Introduction to Testcontainers

Ghislain Gabriëlse

DE TESTERS

Ghislain Gabriëlse

Test Automation Consultant DeTesters

- Woerden, Netherlands 🇳🇱
- 36 years
- Father of 2 minions
- Butler to a cat
- ~12 years of experience
- Builds Tools that simplify complex tasks

Agenda

1. What is Testcontainers?
2. Mocking vs Real Services
3. How It Works
4. Quarkus + Testcontainers in Practice
5. Best Practices

What is Testcontainers?

What is Testcontainers?

Testcontainers is an open-source library that provides lightweight, throwaway instances of real services wrapped in Docker containers.

In a nutshell

- Programmatically start Docker containers from your tests
- Spin up **real** databases, message brokers, browsers, and more
- Containers are created before the test and destroyed after
- Available for Java, Node.js, Python, .NET, Go, Rust, and more

Supported by

- 🐘 **Databases** — PostgreSQL, MySQL, MongoDB, Redis
- 📨 **Messaging** — Kafka, RabbitMQ, Pulsar
- 🌐 **Cloud** — LocalStack (AWS), Azure, GCP emulators
- 🧩 **And much more** — Elasticsearch, Keycloak, Selenium, …

Mocking vs Real Services

Where Mocking Falls Short

😬 The Risks

- **Behavior drift** — Mocks don't update when the real service changes
- **False confidence** — Tests pass, production breaks
- **SQL dialect gaps** — H2 doesn't behave like PostgreSQL
- **Missing edge cases** — Timeouts, connection limits, constraints

🤔 A Common Scenario

```
// This test passes...
when(repo.save(any()))
    .thenReturn(savedEntity);

// But in production, a UNIQUE constraint
// violation causes a 500 error because
// we never tested against a real database.
```

> "Your mocks are only as good as your assumptions about the real system."

Why Testcontainers?

Aspect	Mocking / Stubs	Testcontainers
Fidelity	Simulated behavior	Real service behavior
SQL Dialect	Generic (H2)	Exact production DB
Speed	Very fast	Slightly slower
Confidence	Medium	High
Maintenance	Keep mocks in sync	Self-updating

💡 Testcontainers doesn't replace **all** mocking — it shines for **integration tests**.

Key Benefits

🎯 Test against real services

No more behavior drift — your tests run against the same database engine as production.

🏝️ Complete isolation

Every test run gets fresh containers. No shared state, no flaky tests from leftover data.

🔄 Reproducible everywhere

Works the same on your laptop and in CI. No "works on my machine" problems.

⚡ Easy to set up

A few lines of code to spin up a fully configured PostgreSQL, Kafka, or Redis instance.

📦 No external infrastructure

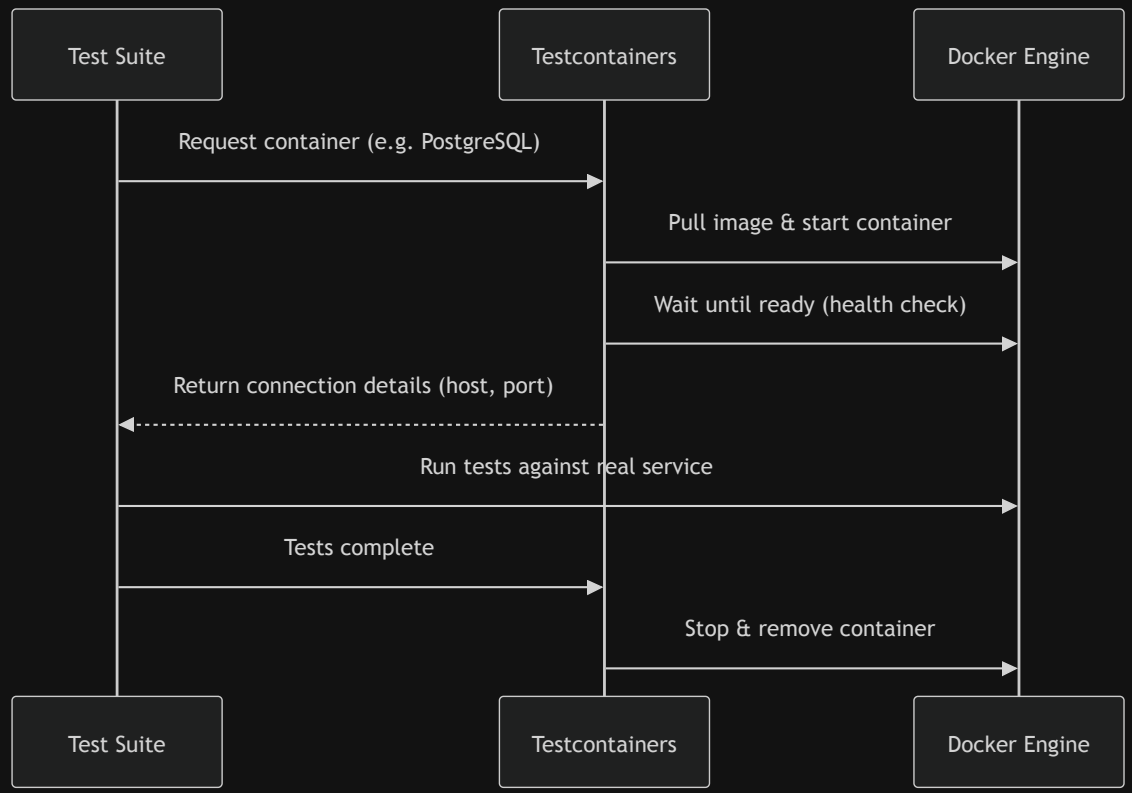
No need to maintain shared test databases or services. Just Docker.

🧹 Automatic cleanup

Containers are automatically stopped and removed when tests finish. The **Ryuk** sidecar ensures nothing leaks.

How It Works

How Testcontainers Works



💡 **Ryuk** — a sidecar container that auto-cleans up all test containers, even if your test process crashes.

Example 1 — PostgreSQL Integration Test

```java
@Testcontainers
class UserRepositoryTest {
    @Container
    static PostgreSQLContainer<?> postgres =
        new PostgreSQLContainer<>("postgres:16-alpine");

    @DynamicPropertySource
    static void configure(DynamicPropertyRegistry registry) {
        registry.add("spring.datasource.url", postgres::getJdbcUrl);
        registry.add("spring.datasource.username", postgres::getUsername);
        registry.add("spring.datasource.password", postgres::getPassword);
    }

    @Test
    void shouldPersistAndRetrieveUser() {
        userRepository.save(new User("Ghislain", "ghislain@example.com"));
        assertThat(userRepository.findByEmail("ghislain@example.com")).isPresent();
    }
}
```

Example 2 — GenericContainer

Use **any** Docker image:

```
@Testcontainers
class CustomServiceTest {
    @Container
    static GenericContainer<?> wiremock =
        new GenericContainer<>(
            "wiremock/wiremock:3.5.4")
            .withExposedPorts(8080)
            .waitingFor(
                Wait.forHttp("/__admin/mappings"));

    @Test
    void shouldConnectToWiremock() {
        var url = "http://" + wiremock.getHost()
            + ":" + wiremock.getMappedPort(8080);
        given().baseUri(url)
            .when().get("/__admin/mappings")
            .then().statusCode(200);
    }
}
```

Quarkus + Testcontainers in Practice

Why Quarkus Makes Testcontainers Shine

🌿 Spring Boot

- Classpath scan + auto-config **at runtime**
- `@SpringBootTest` startup: **5–15s**
- `@MockBean` change → **new context** → new containers
- Container setup: **manual**
- No continuous test mode

⚡ Quarkus

- Bean wiring + config **at build time**
- `@QuarkusTest` startup: **1–3s**
- `@InjectMock` swaps **in-place** → shared context
- Container setup: **automatic** (Dev Services)
- `quarkus:dev` → re-run on save **< 2s**

Closing the Gap in Spring Boot

`@ServiceConnection` (3.1+)

Auto-discovers connection properties:

```
@Container @ServiceConnection
static PostgreSQLContainer<?> postgres =
    new PostgreSQLContainer<>("postgres:16-alpine");
```

Container reuse

```
new PostgreSQLContainer<>("postgres:16-alpine")
    .withReuse(true);
```

Test slicing

Annotation	Loads only
`@DataJpaTest`	JPA + DB
`@WebMvcTest`	Controllers
`@JsonTest`	Serialization

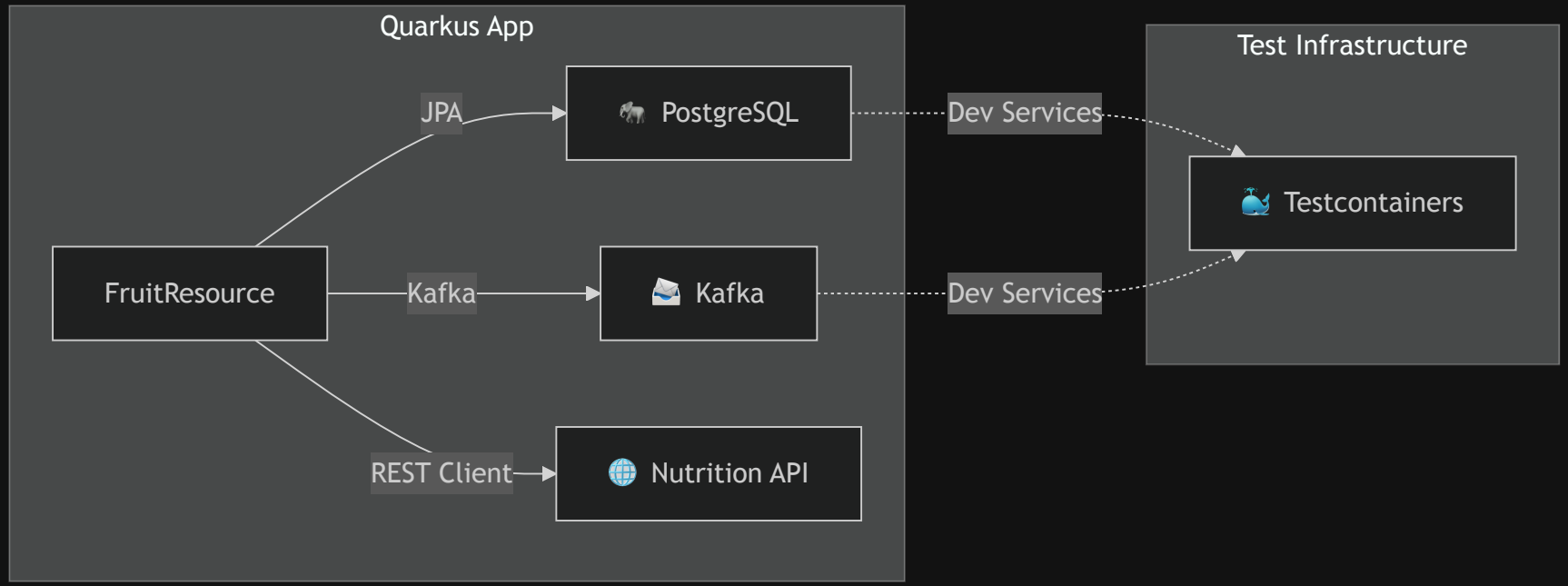
Avoid context pollution

- ❌ Unique `@MockBean` combo → new context
- ❌ `@DirtiesContext` → full rebuild
- ✅ Group tests with same mock setup

💡 Spring Boot can get close — but you **opt in** to what Quarkus provides **by default**.

The Demo Application

A simple **Fruit CRUD API** built with Quarkus, demonstrating two test levels.



Database — Panache ORM + Flyway

Messaging — Kafka events on CRUD

REST Client — External nutrition API

Quarkus Dev Services — Zero Config Containers

In Quarkus, **Dev Services** automatically starts containers — no manual setup needed:

```
# Just declare the DB kind — Quarkus starts PostgreSQL automatically
quarkus.datasource.db-kind=postgresql

# Flyway runs migrations against the Dev Services container
quarkus.flyway.migrate-at-start=true

# Kafka — Quarkus auto-starts a Redpanda container
mp.messaging.outgoing.fruit-events-out.connector=smallrye-kafka
```

💡 No container declaration, no port mapping, no JDBC URL. Quarkus detects the dependencies and starts what's needed.

Dev Services starts: PostgreSQL 16 + Redpanda (Kafka) — both via Testcontainers under the hood.

The Testing Pyramid — Two Levels

🏎️ Unit Test

`@InjectMock` replaces CDI beans

- Kafka producer → mocked
- REST client → mocked
- Database → real (Dev Services)
- ⚡ Fast feedback on logic

🔗 Integration Test

Dev Services provides all containers

- Kafka → real (Redpanda)
- Database → real (PostgreSQL)
- REST client → not tested
- 🎯 Full stack validation

Level 1 — Unit Test with `@InjectMock`

```
        given()
            .contentType("application/json")
            .body("{\"name\": \"Grape\", \"description\": \"Small and round\"}")
        .when().post("/fruits")
        .then()
            .statusCode(201)
            .body("name", is("Grape"));

        Mockito.verify(eventProducer).send(any(FruitEvent.class));
    }

    @Test
    void getDetailsShouldCallNutritionClient() {
        Mockito.when(nutritionClient.getNutrition(eq("apple")))
            .thenReturn(new NutritionInfo(95, "19g", "4.4g"));

        given().when().get("/fruits/1/details")
        .then()
            .statusCode(200)
            .body("nutrition.calories", is(95));
    }
}
```

Level 2 — Integration Test (Real Containers)

```
    @Test
    void shouldListSeededFruits() {
        // Flyway runs V1 + V2 migrations → Apple, Banana, Cherry exist
        given()
            .when().get("/fruits")
            .then()
                .statusCode(200)
                .body("size()", greaterThanOrEqualTo(3))
                .body("name", hasItems("Apple", "Banana", "Cherry"));
    }

    @Test
    void shouldCreateAndFetchFruit() {
        int id = given()
            .contentType("application/json")
            .body("{\"name\": \"Mango\", \"description\": \"Tropical and juicy\"}")
            .when().post("/fruits")
            .then().statusCode(201).extract().path("id");

        given().when().get("/fruits/" + id)
            .then().statusCode(200).body("name", is("Mango"));
    }
}
```

Results — 10 Tests, 2 Levels, Real Confidence

Test Class	Level	Tests
`UnitTest`	Unit 🐘	3
`ResourceTest`	Integration 🐘 📨	7

Key takeaways

- ⚡ Mock what you isolate, real DB is free
- 🎯 Zero config via Dev Services
- 🏗️ Real containers = real confidence
- 🧹 All cleanup is automatic (Ryuk)
- 🔄 Reproducible on any machine with Docker

`mvn test` → **10 tests** ✅ **BUILD SUCCESS** (26s)

Best Practices

Tips & Getting Started

✅ Do

- **Reuse containers** across tests (`static` + `@Container`)
- **Use** `@DynamicPropertySource` — don't hardcode ports
- **Pin image tags** (e.g. `postgres:16-alpine`)
- **Use** `.waitingFor()` strategies

❌ Avoid

- New container per test method — it's slow
- Fixed ports — use `getMappedPort()` instead
- Replacing **all** mocks — unit tests are still valuable

🚀 Getting Started

```
<dependency>
    <groupId>org.testcontainers</groupId>
    <artifactId>postgresql</artifactId>
    <version>1.20.4</version>
    <scope>test</scope>
</dependency>
```

Prerequisites: Docker + JUnit 5. That's it! 🎉

📚 testcontainers.com | 💻 github.com/testcontainers

Thank You!

Questions? 🙋

📚 testcontainers.com — Official documentation
💻 github.com/testcontainers — Source code & examples
🦍 testcontainers.com/modules — Browse all available modules

DE TESTERS